Lab Report: Django Web Application Core Concepts Implementation

Name: Sunil Lamichhane
Roll: 080BCT090
Subject: Web Technology

Lab: Django Framework — Requests, Forms, Sessions, Routing, Middleware, ORM, Auth

---

Table of Contents

1. Brief Description of the Project
2. MVT Architecture
3. Project Structure
4. Code — Major Components
   - 4.1 settings.py
   - 4.2 models.py
   - 4.3 forms.py
   - 4.4 views.py
   - 4.5 urls.py
   - 4.6 middleware.py
   - 4.7 Templates
5. Concepts Covered
6. GitHub Repository
7. Output Screenshots
8. Conclusion

---

1. Brief Description of the Project

This project is a **Blog & Task Manager Web Application** built with Django. It is designed as a single project that demonstrates all the major concepts of Django development in one place.

The application allows users to:

- Register and log in (Authentication & Authorization)
- Create, read, update, and delete blog posts (CRUD + ORM)

- Submit and manage tasks using forms with session-based feedback
- Navigate across multiple pages via URL routing
- Experience middleware-level logging and error handling transparently

The project uses **SQLite** as its relational database (via Django ORM) and also demonstrates how a **NoSQL** (MongoDB via `djongo` or `pymongo`) integration would be structured for comparison.

Tech Stack:

- Python 3.11
- Django 4.2
- SQLite (Relational)
- MongoDB (NoSQL — conceptual integration shown)
- Bootstrap 5 (for templates)

---

2. MVT Architecture

Django follows the **MVT (Model-View-Template)** pattern, which is Django's interpretation of the classic MVC pattern.

```
Request
  |
  ▼
urls.py  ──────────────────────────────────────────────────────►  404 Handler
  | (URL Router — maps URL to View)
  ▼
middleware.py
  | (Logging, Security, Session, Auth checks)
  ▼
views.py  (Controller logic)
  |        |
  |        ▼
  |      models.py  ◄──────►  Database (SQLite / MongoDB)
  |      (ORM queries)
  |
  ▼
templates/  (HTML rendered with context data)
  |
  ▼
Response → Browser
```

MVT Layer	Django Component	Responsibility
Model	`models.py`	Defines data structure, handles DB operations via ORM
View	`views.py`	Handles request logic, processes forms, calls models, returns response
Template	`templates/*.html`	Renders the HTML with data passed from the view
URL Router	`urls.py`	Maps incoming URL patterns to the correct view function
Middleware	`middleware.py`	Intercepts every request/response for logging, auth, security

Difference from MVC:
In classic MVC, the Controller handles routing and logic. In Django's MVT, the **View** acts as the controller, the **Template** acts as the View (presentation), and the URL router acts as the dispatcher.

---

3. Project Structure

```
django_lab/
│
├── manage.py
├── django_lab/                  # Project config
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
│
├── blog/                        # Main app
│   ├── __init__.py
│   ├── models.py                # Post, Task models (ORM)
│   ├── views.py                 # All view functions
│   ├── forms.py                 # Django Forms
│   ├── urls.py                  # App-level URL patterns
│   ├── middleware.py            # Custom middleware
│   ├── admin.py                 # Admin registration
│   └── templates/
│       └── blog/
│           ├── base.html        # Base layout
```

```
│           ├── home.html        # Home page
│           ├── post_list.html   # All posts
│           ├── post_detail.html # Single post
│           ├── post_form.html   # Create/edit post
│           ├── task_form.html   # Task form with session
│           ├── login.html       # Login page
│           └── register.html    # Register page
│
├── static/                      # CSS, JS, images
├── db.sqlite3                   # SQLite database (auto-created)
└── requirements.txt
```

---

4. Code — Major Components

4.1 settings.py

```python
# django_lab/settings.py

from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-your-secret-key-here'

DEBUG = True

ALLOWED_HOSTS = ['*']

# ------------------------------------------------------------
# Installed Apps
# ------------------------------------------------------------
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',          # Built-in auth system
    'django.contrib.contenttypes',
    'django.contrib.sessions',      # Session framework
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'blog',                         # Our custom app
```

```
]

# ------------------------------------------------------------
# Middleware Stack
# Order matters — each middleware wraps the next one.
# Request flows top-to-bottom, Response flows bottom-to-top.
# ------------------------------------------------------------
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',        # Security
headers
    'django.contrib.sessions.middleware.SessionMiddleware', # Session
handling
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',            # CSRF
protection
    'django.contrib.auth.middleware.AuthenticationMiddleware', # Attaches
request.user
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'blog.middleware.RequestLoggerMiddleware',              # Our custom
middleware
    'blog.middleware.LoginRequiredMiddleware',              # Our custom
auth guard
]

ROOT_URLCONF = 'django_lab.urls'

# ------------------------------------------------------------
# Templates
# ------------------------------------------------------------
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
```

```
]

# ------------------------------------------------------------
# Database - Relational (SQLite via Django ORM)
# ------------------------------------------------------------
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Session configuration
SESSION_ENGINE = 'django.contrib.sessions.backends.db'  # Store sessions in
DB
SESSION_COOKIE_AGE = 1800       # Session expires after 30 minutes
SESSION_COOKIE_HTTPONLY = True  # Prevent JavaScript access to session
cookie
SESSION_COOKIE_SECURE = False   # Set True in production (HTTPS only)

# Authentication redirect
LOGIN_URL = '/login/'
LOGIN_REDIRECT_URL = '/'
LOGOUT_REDIRECT_URL = '/login/'

# Static files
STATIC_URL = '/static/'

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

4.2 models.py

```
# blog/models.py
# Models define the database schema.
# Django ORM translates these Python classes into SQL tables automatically.

from django.db import models
from django.contrib.auth.models import User
from django.utils import timezone
```

```python
class Post(models.Model):
    """
    Relational model for a blog post.
    Django ORM will create a 'blog_post' table in SQLite with these columns.
    """
    title   = models.CharField(max_length=200)
    content = models.TextField()
    author  = models.ForeignKey(
        User,
        on_delete=models.CASCADE,   # Delete posts when user is deleted
        related_name='posts'
    )
    created_at = models.DateTimeField(default=timezone.now)
    updated_at = models.DateTimeField(auto_now=True)
    is_published = models.BooleanField(default=True)

    class Meta:
        ordering = ['-created_at']  # Newest first by default

    def __str__(self):
        return self.title


class Task(models.Model):
    """
    Relational model for a user task.
    Demonstrates a second table and a ForeignKey relationship.
    """
    STATUS_CHOICES = [
        ('pending',     'Pending'),
        ('in_progress', 'In Progress'),
        ('done',        'Done'),
    ]

    title       = models.CharField(max_length=150)
    description = models.TextField(blank=True)
    status      = models.CharField(max_length=20, choices=STATUS_CHOICES,
default='pending')
    owner       = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='tasks')
    due_date    = models.DateField(null=True, blank=True)
    created_at  = models.DateTimeField(auto_now_add=True)
```

```
    def __str__(self):
        return f"{self.title} [{self.status}]"


# ------------------------------------------------------------
# NoSQL Equivalent (MongoDB / pymongo — no ORM)
# This is how you would insert/query the same data in MongoDB:
#
# from pymongo import MongoClient
#
# client = MongoClient("mongodb://localhost:27017/")
# db     = client["django_lab_db"]
#
# # Insert a post (equivalent to Post.objects.create(...))
# db.posts.insert_one({
#     "title":      "My First Post",
#     "content":    "Hello MongoDB",
#     "author":     "sunil",
#     "created_at": datetime.utcnow(),
# })
#
# # Query all posts (equivalent to Post.objects.all())
# posts = list(db.posts.find())
#
# # Update (equivalent to Post.objects.filter(id=1).update(...))
# db.posts.update_one({"_id": post_id}, {"$set": {"title": "Updated"}})
#
# # Delete (equivalent to Post.objects.get(id=1).delete())
# db.posts.delete_one({"_id": post_id})
# ------------------------------------------------------------
```

4.3 forms.py

```
# blog/forms.py
# Django Forms handle validation, rendering, and data cleaning
automatically.

from django import forms
from django.contrib.auth.models import User
from django.contrib.auth.forms import UserCreationForm
from .models import Post, Task
```

```python
class PostForm(forms.ModelForm):
    """
    ModelForm automatically generates form fields from the Post model.
    It also handles validation rules defined on the model (max_length, etc.)
    """
    class Meta:
        model   = Post
        fields  = ['title', 'content', 'is_published']
        widgets = {
            'title':   forms.TextInput(attrs={
                'class': 'form-control',
                'placeholder': 'Enter post title'
            }),
            'content': forms.Textarea(attrs={
                'class': 'form-control',
                'rows': 6,
                'placeholder': 'Write your post content here...'
            }),
            'is_published': forms.CheckboxInput(attrs={'class': 'form-check-
input'}),
        }


class TaskForm(forms.ModelForm):
    """
    Form for creating and editing tasks.
    Demonstrates form with dropdown choices and date picker.
    """
    class Meta:
        model  = Task
        fields = ['title', 'description', 'status', 'due_date']
        widgets = {
            'title': forms.TextInput(attrs={'class': 'form-control'}),
            'description': forms.Textarea(attrs={'class': 'form-control',
'rows': 3}),
            'status':   forms.Select(attrs={'class': 'form-select'}),
            'due_date': forms.DateInput(attrs={'class': 'form-control',
'type': 'date'}),
        }
```

```python
class RegisterForm(UserCreationForm):
    """
    Extends Django's built-in UserCreationForm to add an email field.
    """
    email = forms.EmailField(
        required=True,
        widget=forms.EmailInput(attrs={'class': 'form-control'})
    )

    class Meta:
        model  = User
        fields = ['username', 'email']

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Apply Bootstrap classes to all fields
        for field in self.fields.values():
            field.widget.attrs.setdefault('class', 'form-control')
```

4.4 views.py

```python
# blog/views.py
# Views handle the request, call models, process forms, and return
responses.
# This is the "Controller" layer in Django's MVT pattern.

from django.shortcuts import render, get_object_or_404, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.http import HttpResponse, JsonResponse
from .models import Post, Task
from .forms import PostForm, TaskForm, RegisterForm


# ------------------------------------------------------------
# 1. Handling Requests & Responses
# ------------------------------------------------------------

def home(request):
    """
```

```
    Demonstrates basic request handling and response.
    request.method tells us if it is GET or POST.
    render() returns an HttpResponse with a rendered template.
    """
    # Access request metadata
    user_agent = request.META.get('HTTP_USER_AGENT', 'Unknown')
    client_ip  = request.META.get('REMOTE_ADDR', 'Unknown')

    context = {
        'user_agent': user_agent,
        'client_ip':  client_ip,
        'user':       request.user,
    }
    return render(request, 'blog/home.html', context)


def api_posts(request):
    """
    Returns posts as JSON — demonstrates JsonResponse for API-style
responses.
    """
    posts = list(Post.objects.values('id', 'title', 'created_at'))
    return JsonResponse({'posts': posts})


# ------------------------------------------------------------
# 2. CRUD Operations (Post)
# ------------------------------------------------------------

def post_list(request):
    """READ — Fetch all published posts from the database via ORM."""
    posts = Post.objects.filter(is_published=True).select_related('author')
    return render(request, 'blog/post_list.html', {'posts': posts})


def post_detail(request, pk):
    """READ (single) — get_object_or_404 handles the 404 automatically."""
    post = get_object_or_404(Post, pk=pk, is_published=True)
    return render(request, 'blog/post_detail.html', {'post': post})


@login_required
def post_create(request):
```

```
    """
    CREATE - Handles both GET (show blank form) and POST (save form data).
    Demonstrates ModelForm processing and redirect after success.
    """
    if request.method == 'POST':
        form = PostForm(request.POST)
        if form.is_valid():
            post = form.save(commit=False)  # Don't save to DB yet
            post.author = request.user      # Attach the logged-in user
            post.save()                     # Now save to DB
            messages.success(request, 'Post created successfully!')
            return redirect('post_detail', pk=post.pk)
    else:
        form = PostForm()   # Empty form for GET request

    return render(request, 'blog/post_form.html', {'form': form, 'action':
'Create'})


@login_required
def post_update(request, pk):
    """UPDATE - Pre-populate form with existing instance."""
    post = get_object_or_404(Post, pk=pk, author=request.user)

    if request.method == 'POST':
        form = PostForm(request.POST, instance=post)  # Bind form to
existing object
        if form.is_valid():
            form.save()
            messages.success(request, 'Post updated successfully!')
            return redirect('post_detail', pk=post.pk)
    else:
        form = PostForm(instance=post)  # Pre-fill form with current data

    return render(request, 'blog/post_form.html', {'form': form, 'action':
'Update'})


@login_required
def post_delete(request, pk):
    """DELETE - Confirm on GET, delete on POST."""
    post = get_object_or_404(Post, pk=pk, author=request.user)
```

```python
    if request.method == 'POST':
        post.delete()
        messages.success(request, 'Post deleted.')
        return redirect('post_list')

    return render(request, 'blog/post_confirm_delete.html', {'post': post})


# ------------------------------------------------------------
# 3. Form Data Handling & Sessions
# ------------------------------------------------------------

@login_required
def task_form(request):
    """
    Demonstrates form handling + session usage.
    Sessions persist data across requests for the same user.
    """
    if request.method == 'POST':
        form = TaskForm(request.POST)
        if form.is_valid():
            task = form.save(commit=False)
            task.owner = request.user
            task.save()

            # Store feedback in session (persists until read)
            request.session['last_task'] = task.title

            # Django messages framework (uses session internally)
            messages.success(request, f'Task "{task.title}" created!')
            return redirect('task_list')
    else:
        form = TaskForm()

    # Read last task title from session
    last_task = request.session.get('last_task', None)

    return render(request, 'blog/task_form.html', {
        'form': form,
        'last_task': last_task
    })
```

```python
@login_required
def task_list(request):
    """List all tasks owned by the current user."""
    tasks = Task.objects.filter(owner=request.user).order_by('status')
    return render(request, 'blog/task_list.html', {'tasks': tasks})


# ------------------------------------------------------------
# 4. Authentication & Authorization
# ------------------------------------------------------------

def register_view(request):
    """
    User Registration — creates a new User record using Django's auth
system.
    """
    if request.user.is_authenticated:
        return redirect('home')

    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)    # Log the user in immediately after
registration
            messages.success(request, f'Account created! Welcome,
{user.username}.')
            return redirect('home')
    else:
        form = RegisterForm()

    return render(request, 'blog/register.html', {'form': form})


def login_view(request):
    """
    User Login — authenticate() checks credentials, login() creates the
session.
    """
    if request.user.is_authenticated:
        return redirect('home')

    if request.method == 'POST':
```

```
        username = request.POST.get('username')
        password = request.POST.get('password')

        # authenticate() returns User object if credentials are valid, else
None
        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)    # Creates session: sets SESSION_ID
cookie
            messages.success(request, f'Welcome back, {user.username}!')

            # Redirect to the page the user was trying to access, or home
            next_url = request.GET.get('next', '/')
            return redirect(next_url)
        else:
            messages.error(request, 'Invalid username or password.')

    return render(request, 'blog/login.html')


@login_required
def logout_view(request):
    """
    Logout - clears the session and logs the user out.
    Uses POST to protect against CSRF-based forced logout.
    """
    if request.method == 'POST':
        logout(request)     # Deletes the session from the database
        messages.info(request, 'You have been logged out.')
        return redirect('login')

    return render(request, 'blog/logout_confirm.html')
```

4.5 urls.py

```
# django_lab/urls.py  (Project-level URL configuration)

from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('blog.urls')),     # Delegate all other URLs to the
blog app
]
```

```
# blog/urls.py  (App-level URL configuration)
# Each path() maps a URL pattern to a view function.
# Named URLs (name=) allow reverse URL lookups in templates with {% url
'name' %}

from django.urls import path
from . import views

urlpatterns = [
    # Home
    path('',                    views.home,          name='home'),

    # Posts (CRUD)
    path('posts/',              views.post_list,     name='post_list'),
    path('posts/<int:pk>/',     views.post_detail,   name='post_detail'),
    path('posts/create/',       views.post_create,   name='post_create'),
    path('posts/<int:pk>/edit/',views.post_update,   name='post_update'),
    path('posts/<int:pk>/delete/', views.post_delete, name='post_delete'),

    # Tasks (Forms + Sessions)
    path('tasks/',              views.task_list,     name='task_list'),
    path('tasks/new/',          views.task_form,     name='task_form'),

    # Authentication
    path('register/',           views.register_view, name='register'),
    path('login/',              views.login_view,    name='login'),
    path('logout/',             views.logout_view,   name='logout'),

    # API endpoint (JSON response demo)
    path('api/posts/',          views.api_posts,     name='api_posts'),
]
```

4.6 middleware.py

```python
# blog/middleware.py
# Middleware is a hook system that processes every request and response.
# Each class must implement __init__(get_response) and __call__(request).

import logging
import time
from django.http import HttpResponseForbidden
from django.shortcuts import redirect
from django.conf import settings

# Use Python's standard logging module
logger = logging.getLogger(__name__)


class RequestLoggerMiddleware:
    """
    Logs every incoming request: method, path, user, and response time.
    Demonstrates middleware for logging.
    """

    def __init__(self, get_response):
        # Called once when the server starts
        self.get_response = get_response

    def __call__(self, request):
        # --- Code here runs BEFORE the view ---
        start_time = time.time()

        user = request.user if hasattr(request, 'user') else 'AnonymousUser'

        logger.info(
            f"[REQUEST]  {request.method} {request.path} | "
            f"User: {user} | IP: {request.META.get('REMOTE_ADDR')}"
        )

        # Pass the request to the next middleware / view
        response = self.get_response(request)

        # --- Code here runs AFTER the view ---
        duration = (time.time() - start_time) * 1000  # Convert to ms

        logger.info(
            f"[RESPONSE] {request.method} {request.path} | "
```

```
            f"Status: {response.status_code} | Time: {duration:.2f}ms"
        )

        return response


class LoginRequiredMiddleware:
    """
    Redirects unauthenticated users to the login page for protected paths.
    Demonstrates middleware for authorization / security.
    """

    # URLs that do NOT require login
    PUBLIC_URLS = ['/login/', '/register/', '/admin/', '/api/']

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # Check if this URL requires authentication
        is_public = any(
            request.path.startswith(url) for url in self.PUBLIC_URLS
        )

        if not is_public and not request.user.is_authenticated:
            return redirect(f"{settings.LOGIN_URL}?next={request.path}")

        return self.get_response(request)


class SecurityHeadersMiddleware:
    """
    Adds security-related HTTP headers to every response.
    Demonstrates middleware for security hardening.
    """

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)

        # Prevent MIME type sniffing
```

```
        response['X-Content-Type-Options'] = 'nosniff'

        # Prevent clickjacking
        response['X-Frame-Options'] = 'DENY'

        # Force HTTPS (in production)
        response['Strict-Transport-Security'] = 'max-age=31536000;
includeSubDomains'

        # Basic Content Security Policy
        response['Content-Security-Policy'] = "default-src 'self'"

        return response


class ErrorHandlerMiddleware:
    """
    Catches unhandled exceptions globally and logs them.
    Demonstrates middleware for error handling.
    """

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        return self.get_response(request)

    def process_exception(self, request, exception):
        """
        process_exception() is called automatically by Django
        whenever a view raises an unhandled exception.
        """
        logger.error(
            f"[ERROR] Unhandled exception on {request.path} | "
            f"User: {request.user} | Error: {str(exception)}",
            exc_info=True  # Includes full traceback in logs
        )
        # Return None to let Django's default error handling continue
        return None
```

4.7 Templates

base.html

```
<!-- blog/templates/blog/base.html -->
<!-- Base template - all other templates extend this using {% extends %} -->

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>{% block title %}Django Project{% endblock %}</title>
    <link

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.cs
s"
      rel="stylesheet"
    />
  </head>
  <body>
    <!-- Navigation Bar -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
      <div class="container">
        <a class="navbar-brand" href="{% url 'home' %}">Django Lab</a>
        <div class="navbar-nav ms-auto">
          <a class="nav-link" href="{% url 'post_list' %}">Posts</a>
          <a class="nav-link" href="{% url 'task_list' %}">Tasks</a>

          {% if user.is_authenticated %}
          <span class="nav-link text-light">Hello, {{ user.username }}
</span>
          <form method="POST" action="{% url 'logout' %}" class="d-inline">
            {% csrf_token %}
            <button type="submit" class="btn btn-sm btn-outline-light ms-2">
              Logout
            </button>
          </form>
          {% else %}
          <a class="nav-link" href="{% url 'login' %}">Login</a>
          <a class="nav-link" href="{% url 'register' %}">Register</a>
          {% endif %}
        </div>
      </div>
    </nav>
```

```
    <!-- Flash Messages -->
    <div class="container mt-3">
      {% for message in messages %}
      <div
        class="alert alert-{{ message.tags }} alert-dismissible fade show"
        role="alert"
      >
        {{ message }}
        <button
          type="button"
          class="btn-close"
          data-bs-dismiss="alert"
        ></button>
      </div>
      {% endfor %}
    </div>

    <!-- Main Content Block -->
    <main class="container my-4">{% block content %}{% endblock %}</main>

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
  </body>
</html>
```

home.html

```
<!-- blog/templates/blog/home.html -->
gi
{% extends 'blog/base.html' %} {% block title %}Home - Django Lab{% endblock
%}
{% block content %}
<div class="row">
  <div class="col-md-8">
    <h1 class="mb-3">Welcome to Django Lab</h1>
    <p class="lead">This project demonstrates core Django concepts.</p>

    {% if user.is_authenticated %}
    <p>Logged in as: <strong>{{ user.username }}</strong></p>
    <a href="{% url 'post_create' %}" class="btn btn-primary"
      >Create New Post</a
```

```
    >
    <a href="{% url 'task_form' %}" class="btn btn-secondary ms-2">New
Task</a>
    {% else %}
    <a href="{% url 'login' %}" class="btn btn-primary">Login</a>
    <a href="{% url 'register' %}" class="btn btn-outline-primary ms-2"
      >Register</a
    >
    {% endif %}
  </div>

  <!-- Request info - demonstrates access to request metadata in templates -
->
  <div class="col-md-4">
    <div class="card bg-light">
      <div class="card-body">
        <h6 class="card-title">Request Info</h6>
        <small>
          <strong>IP:</strong> {{ client_ip }}<br />
          <strong>Browser:</strong> {{ user_agent|truncatechars:60 }}
        </small>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

post_list.html

```
<!-- blog/templates/blog/post_list.html -->

{% extends 'blog/base.html' %} {% block title %}All Posts{% endblock %} {%
block
content %}
<div class="d-flex justify-content-between align-items-center mb-4">
  <h2>Blog Posts</h2>
  {% if user.is_authenticated %}
  <a href="{% url 'post_create' %}" class="btn btn-primary">+ New Post</a>
  {% endif %}
</div>

{% if posts %} {% for post in posts %}
<div class="card mb-3">
```

```
  <div class="card-body">
    <h5 class="card-title">
      <a href="{% url 'post_detail' post.pk %}">{{ post.title }}</a>
    </h5>
    <p class="card-text text-muted">
      By {{ post.author.username }} on {{ post.created_at|date:"F j, Y" }}
    </p>
    <p class="card-text">{{ post.content|truncatewords:30 }}</p>

    {% if user == post.author %}
    <a href="{% url 'post_update' post.pk %}" class="btn btn-sm btn-warning"
      >Edit</a
    >
    <a href="{% url 'post_delete' post.pk %}" class="btn btn-sm btn-danger
ms-1"
      >Delete</a
    >
    {% endif %}
  </div>
</div>
{% endfor %} {% else %}
<p class="text-muted">No posts yet. Be the first to create one!</p>
{% endif %} {% endblock %}
```

post_form.html

```
<!-- blog/templates/blog/post_form.html -->

{% extends 'blog/base.html' %} {% block title %}{{ action }} Post{% endblock
%}
{% block content %}
<div class="row justify-content-center">
  <div class="col-md-8">
    <h2>{{ action }} Post</h2>

    <form method="POST">
      {% csrf_token %}
      <!-- Django CSRF protection token -->

      <div class="mb-3">
        <label class="form-label">Title</label>
        {{ form.title }} {% if form.title.errors %}
        <div class="text-danger small">{{ form.title.errors }}</div>
```

```
        {% endif %}
      </div>

      <div class="mb-3">
        <label class="form-label">Content</label>
        {{ form.content }}
      </div>

      <div class="mb-3 form-check">
        {{ form.is_published }}
        <label class="form-check-label">Published</label>
      </div>

      <button type="submit" class="btn btn-primary">{{ action }}
Post</button>
      <a href="{% url 'post_list' %}" class="btn btn-secondary ms-
2">Cancel</a>
    </form>
  </div>
</div>
{% endblock %}
```

login.html

```
<!-- blog/templates/blog/login.html -->

{% extends 'blog/base.html' %} {% block title %}Login{% endblock %} {% block
content %}
<div class="row justify-content-center">
  <div class="col-md-5">
    <div class="card shadow-sm">
      <div class="card-body p-4">
        <h3 class="mb-4">Login</h3>

        <form method="POST">
          {% csrf_token %}

          <div class="mb-3">
            <label class="form-label">Username</label>
            <input
              type="text"
              name="username"
              class="form-control"
```

```
              required
              autofocus
            />
          </div>

          <div class="mb-3">
            <label class="form-label">Password</label>
            <input
              type="password"
              name="password"
              class="form-control"
              required
            />
          </div>

          <button type="submit" class="btn btn-primary w-100">Login</button>
        </form>

        <hr />
        <p class="text-center mb-0">
          Don't have an account?
          <a href="{% url 'register' %}">Register here</a>
        </p>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

task_form.html

```
<!-- blog/templates/blog/task_form.html -->
<!-- Demonstrates session data displayed in template -->

{% extends 'blog/base.html' %} {% block title %}New Task{% endblock %} {%
block
content %}
<div class="row justify-content-center">
  <div class="col-md-7">
    <h2>Create New Task</h2>

    <!-- Show last created task from session -->
    {% if last_task %}
```

```
    <div class="alert alert-info">
      Your last task was: <strong>{{ last_task }}</strong>
    </div>
    {% endif %}

    <form method="POST">
      {% csrf_token %}

      <div class="mb-3">
        <label class="form-label">Title</label>
        {{ form.title }}
      </div>
      <div class="mb-3">
        <label class="form-label">Description</label>
        {{ form.description }}
      </div>
      <div class="mb-3">
        <label class="form-label">Status</label>
        {{ form.status }}
      </div>
      <div class="mb-3">
        <label class="form-label">Due Date</label>
        {{ form.due_date }}
      </div>

      <button type="submit" class="btn btn-success">Save Task</button>
      <a href="{% url 'task_list' %}" class="btn btn-secondary ms-
2">Cancel</a>
    </form>
  </div>
</div>
{% endblock %}
```

---

5. Concepts Covered

Handling Requests & Responses

Every Django view receives an `HttpRequest` object and must return an `HttpResponse`. The `request` object contains the HTTP method (`GET` / `POST`), query parameters, POST data, headers, session, and the authenticated user. Views return responses using `render()` for HTML, `redirect()` for URL redirects, or `JsonResponse()` for API responses.

Form Data Handling & Sessions

Django Forms (`forms.ModelForm`) automate field rendering, validation, and data cleaning. On `POST` , `form.is_valid()` runs all validators. Sessions (`request.session`) persist key-value data server-side between requests, identified by a cookie sent to the browser. This is used for login state, flash messages, and storing temporary data like the last submitted task.

Routing

URL routing in `urls.py` maps URL patterns to views using `path()` . App-level `urls.py` files keep routing modular. Named URLs (e.g., `name='post_detail'`) allow templates and views to reference URLs symbolically using `{% url 'post_detail' pk %}` , so URLs can be changed in one place without breaking references.

Middleware

Middleware is a chain of hooks that wraps every request and response. Django applies middleware in the order listed in `settings.MIDDLEWARE` . Custom middleware classes implement `__call__` to intercept requests before and after views, enabling centralized logging, authentication guards, and security header injection.

Templating

Django templates use `{{ variable }}` for output and `{% tag %}` for logic. Templates extend a base layout using `{% extends %}` and `{% block %}` , promoting DRY (Don't Repeat Yourself) design. Template filters like `|date` , `|truncatewords` , and `|truncatechars` format data for display.

Database Integration — ORM vs NoSQL

Django ORM maps Python model classes to relational database tables. Queries are written in Python (`Post.objects.filter(...)`) and translated to SQL automatically. This provides database portability and prevents SQL injection. The `models.py` code also shows the equivalent raw MongoDB/pymongo operations to contrast the document-oriented NoSQL approach with the relational ORM approach.

Authentication & Authorization

Django's built-in auth system (`django.contrib.auth`) handles user creation, password hashing (PBKDF2), login sessions, and the `@login_required` decorator for view-level access control. `authenticate()` verifies credentials, `login()` creates a session, and `logout()` destroys it. The `LoginRequiredMiddleware` adds route-level protection globally.

Cookies & Sessions

When a user logs in, Django creates a session record in the database and sends a `sessionid` cookie to the browser. On every subsequent request, Django reads this cookie,

looks up the session, and attaches the user object to `request.user`. Session cookies are configured to be `HttpOnly` and `Secure` in settings.

---

6. GitHub Repository

Repository Link: https://github.com/indeedSunil/django-project-web-lab

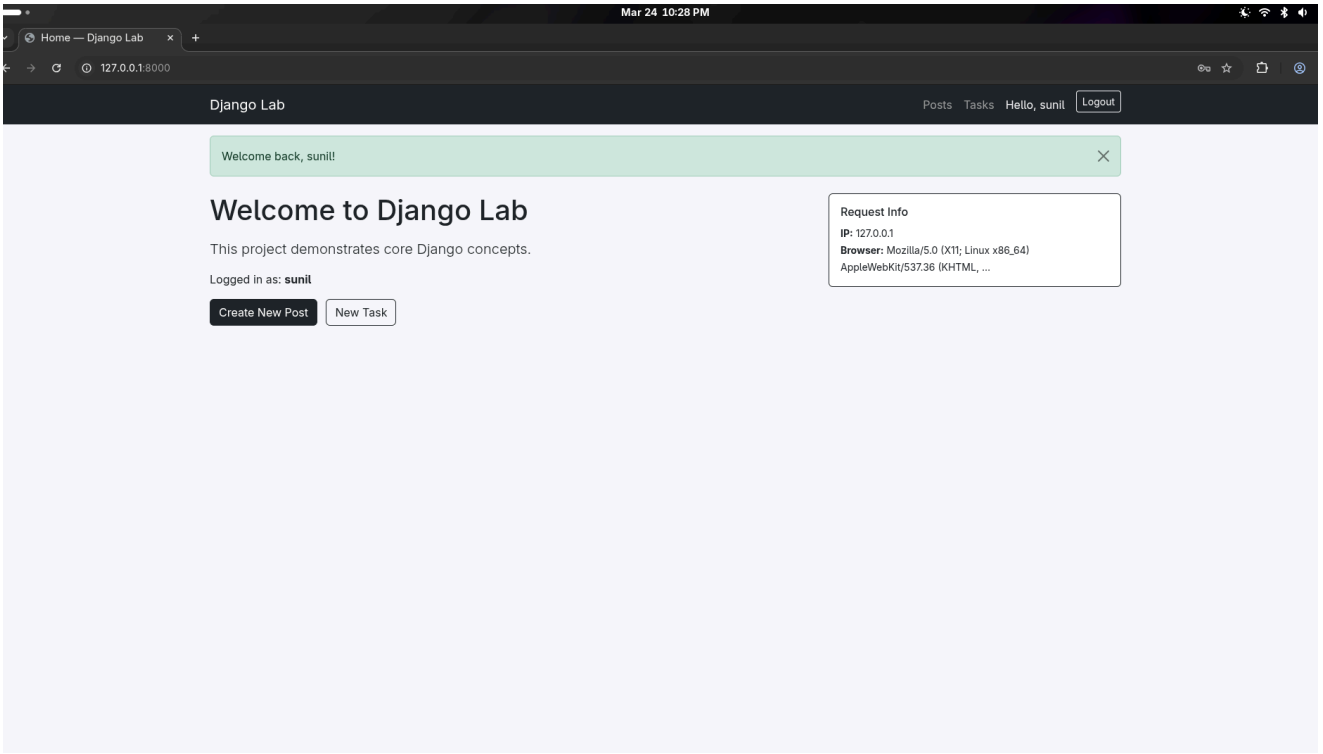
To run the project locally:

```
# Clone the repository
git clone https://github.com/indeedSunil/django-project-web-lab.git
cd django-lab

# Create virtual environment
python -m venv venv
source venv/bin/activate       # On Windows: venv\Scripts\activate

# Install dependencies
pip install django django-bootstrap-v5

# Apply migrations
python manage.py migrate

# Create superuser (for admin panel)
python manage.py createsuperuser

# Run development server
python manage.py runserver
```

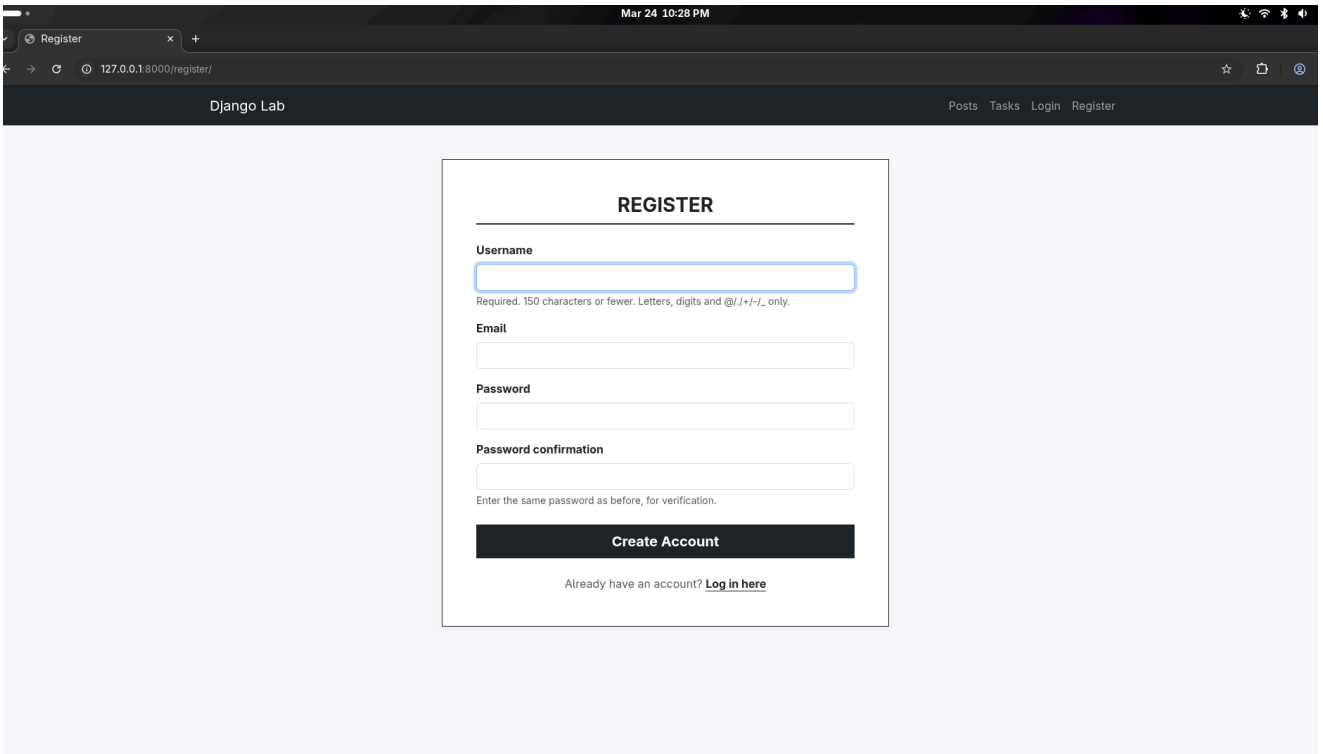
---

7. Output Screenshots

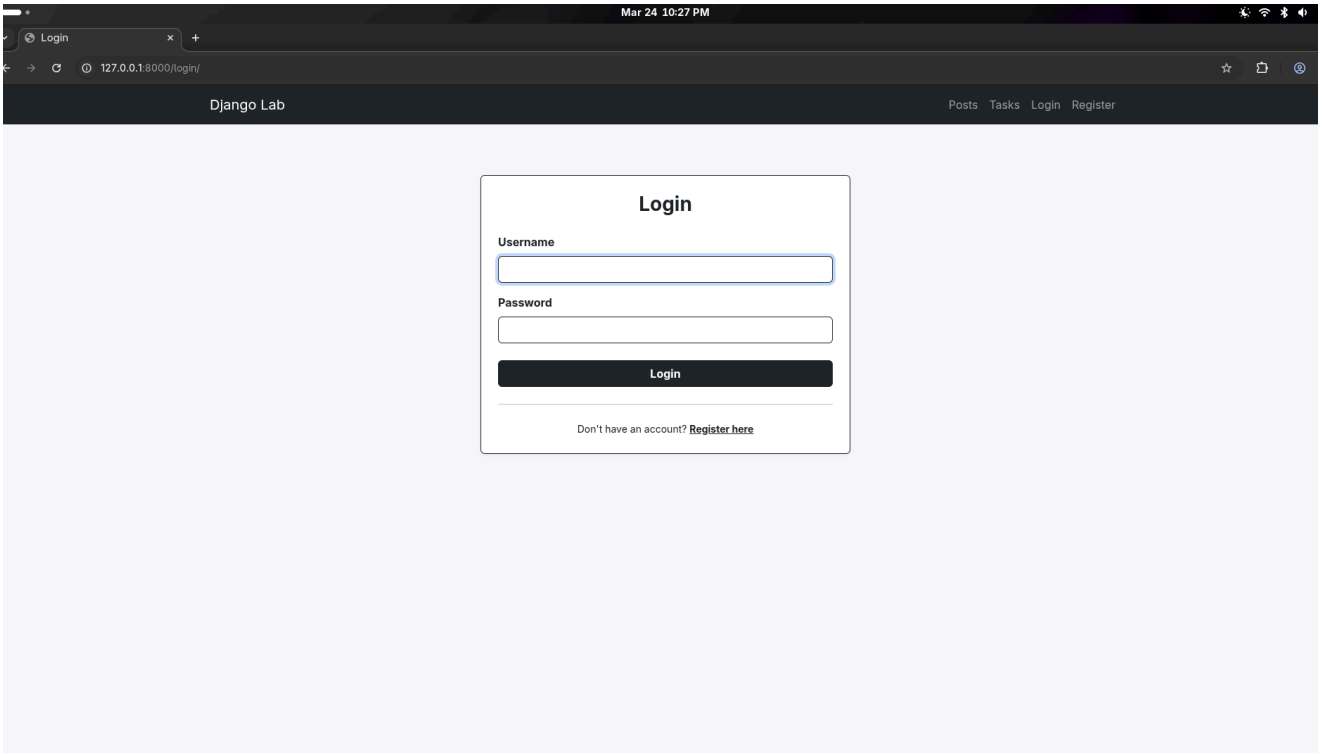
Screenshot 1 — Home Page



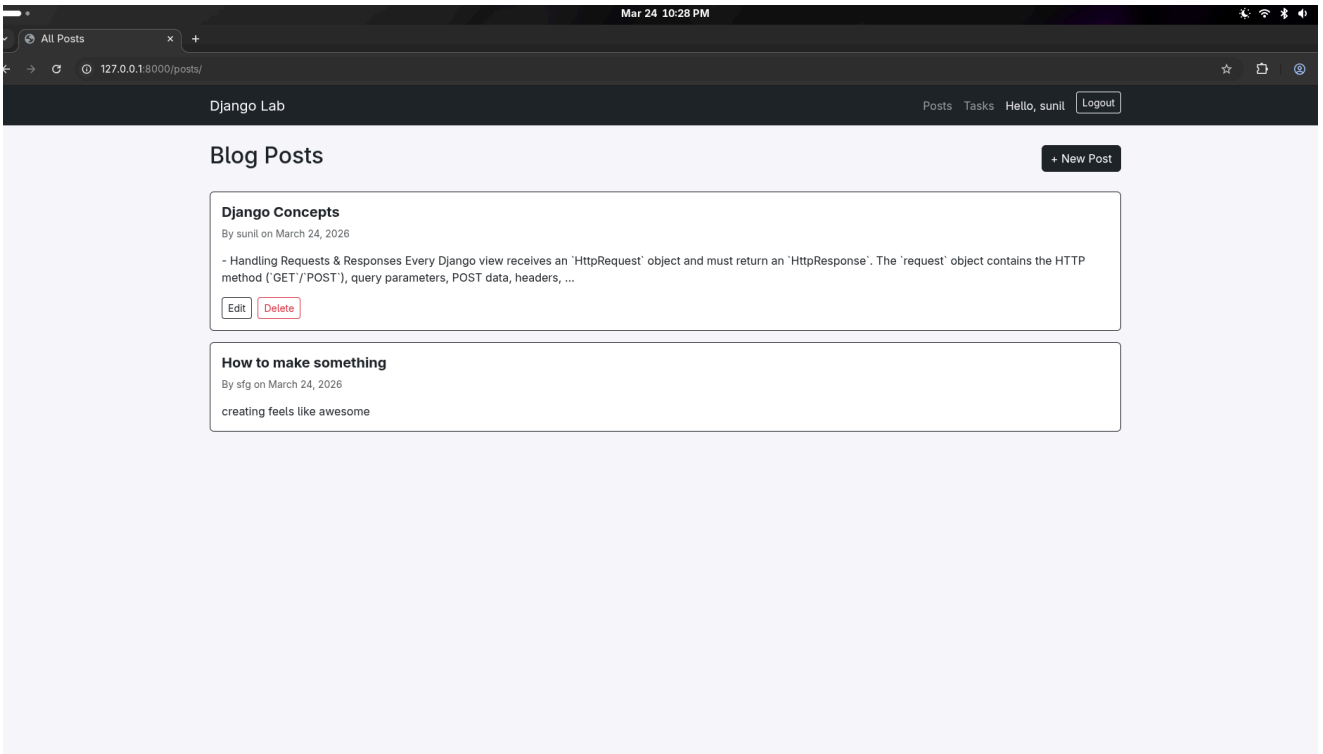
Screenshot 2 — User Registration



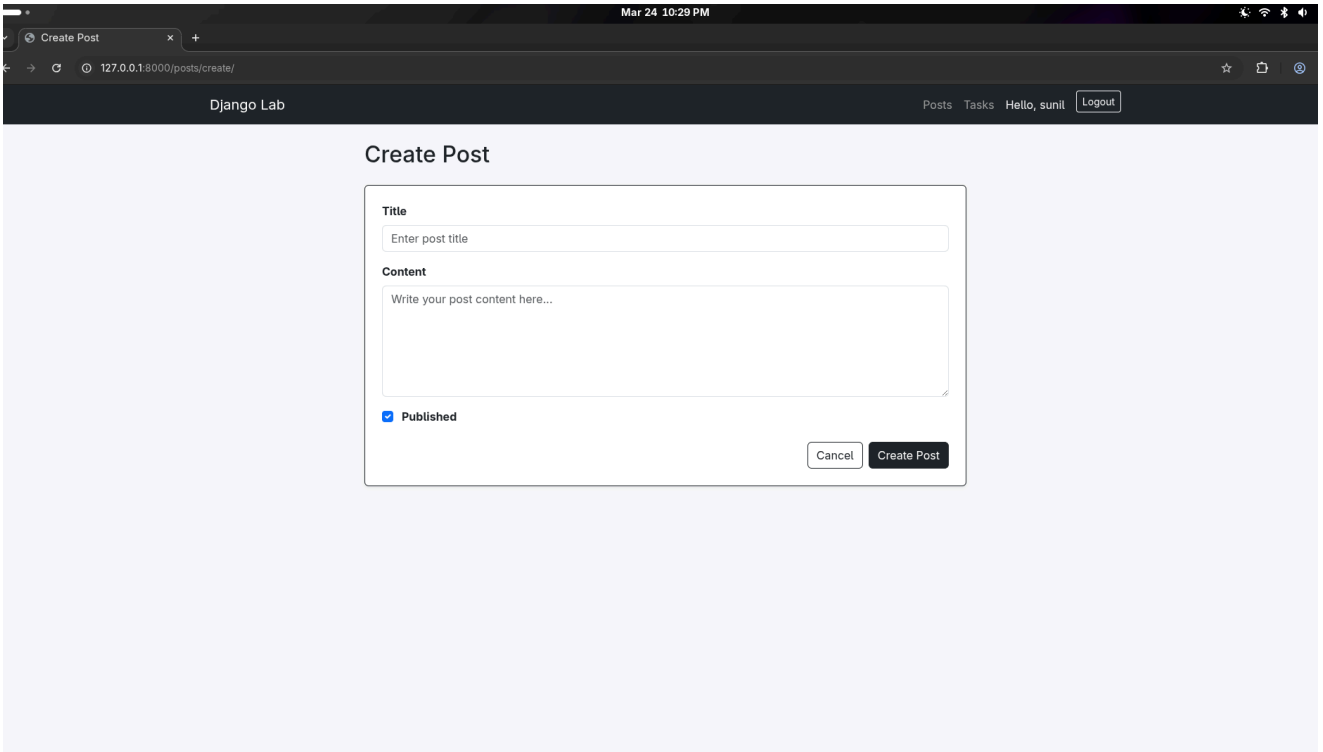
Screenshot 3 — Login Page



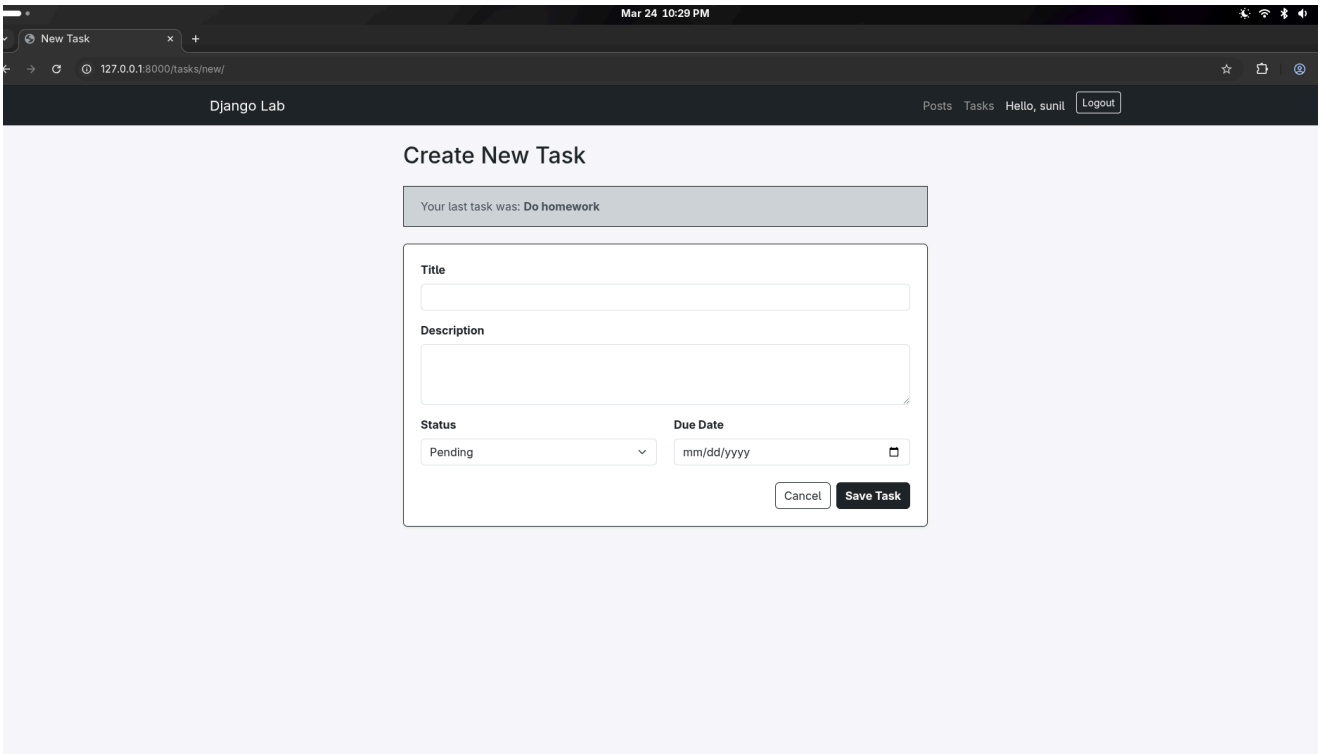
Screenshot 4 — Blog Post List



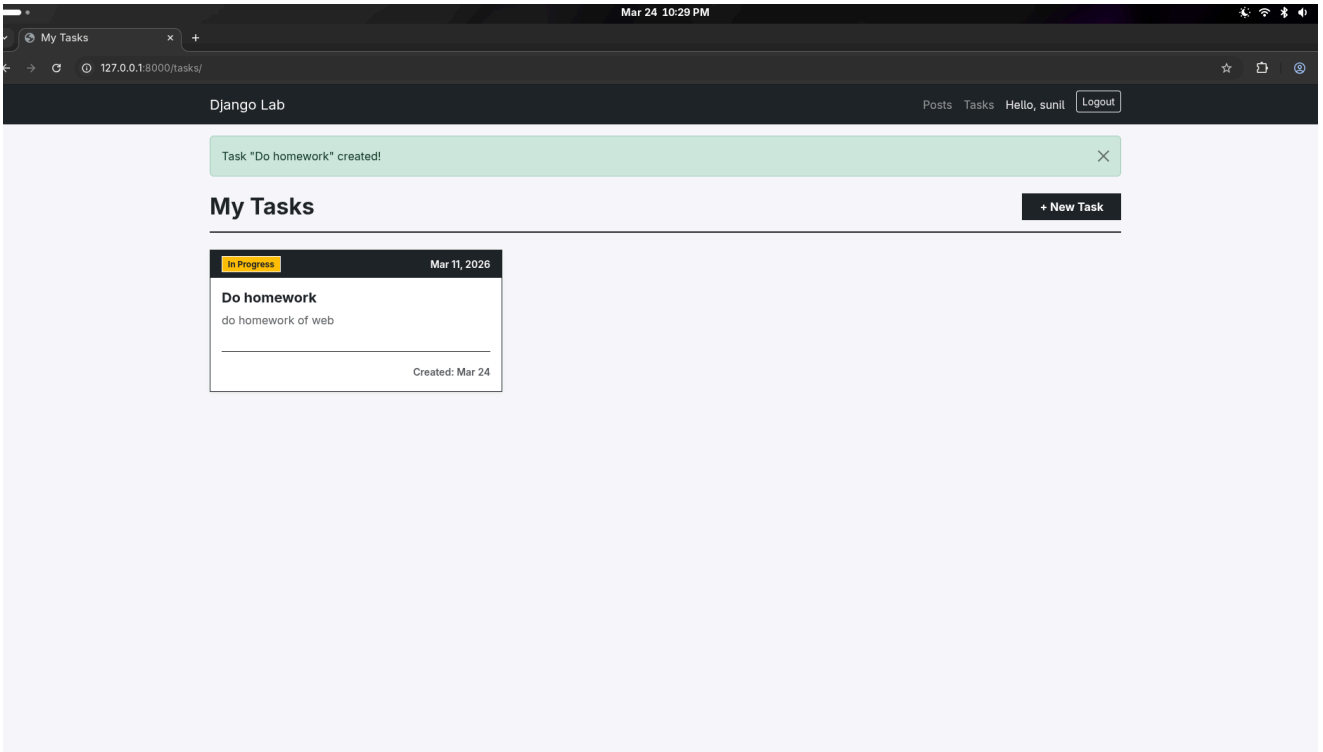
Screenshot 5 — Create Post Form



Screenshot 6 — Create TaskForm



Screenshot 7 — Task Form with Session Data



---

8. Conclusion

This lab successfully demonstrated the core pillars of Django web development in a single cohesive project.